

Certificate Authority (CA)

- 👉 A CA (Certificate Authority) is a trusted entity that issues certificates.
- Examples:
 - Public: Let's Encrypt , DigiCert
 - Internal: HashiCorp Vault
 - ✓ CA ensures: Server identity is valid & Clients can trust connection

Where TLS is Used in Kubernetes?

🧩 Core Components:

- 🔗 API Server (HTTPS for kubectl)
- 💾 etcd (encrypted storage communication)
- 🖥 kubelet ↔ API Server (mTLS)
- ⚙ Controller Manager & Scheduler
- 🌐 Ingress (HTTPS for apps)
- 🔄 Admission Webhooks

Self-Signed Certificate

📌 What is it?

- Certificate signed by itself (not CA)
- ✓ Use Cases: Internal apps & Testing environments
- ✗ Problems: Browsers don't trust & No third-party validation

Certificate Expiry Monitoring

- Use cert-manager + Prometheus alerts & cert-manager metrics
- Set alerts before expiry

✗ What Happens if Certificate Expires?

- TLS connection fails 🚫
- Browser shows warning ⚠

- Clients reject connection
 - ✓ Solution: Use `cert-manager auto-renewal`
-

🔥 Auto TLS with cert-manager

- 👉 Best practice (Production)
- Steps:
 - i. Install cert-manager
 - ii. Create ClusterIssuer
 - iii. Annotate Ingress

```
cert-manager.io/cluster-issuer: letsencrypt-prod
```

4. ✓ Automatic certificate creation & renewal

🏢 Certificate Authority (CA)

- A CA is a trusted entity that signs certificates.
- 🌐 Examples:
 - Let's Encrypt
 - DigiCert
 - HashiCorp Vault

🤝 TLS Handshake Process:

- Exchange keys
- Verify certificates
- Establish secure connection

↻ Certificate Renewal / Rotation

🔧 kubeadm:

```
kubeadm certs renew <component>
```

- cert-manager: ✓ Auto-renew before expiry (Auto-renewal enabled)

- Manual: ✓ Regenerate → Update secrets → Restart pods

🔑 Self-Signed vs CA-Signed

🧩 Type	🔒 Trust Level	🧠 How It Works	💡 Use Case
🔧 Self-Signed	⚠️ Not trusted by default	👉 Certificate signs itself (no external authority)	Dev, testing, internal clusters
🌐 CA-Signed	✅ Trusted globally	👉 Signed by a trusted Certificate Authority (CA)	Public websites, production apps

🕸️ Kubernetes Control Plane Communication

📌 Overview

In Kubernetes, the control plane components communicate primarily through the API Server, which acts as the central management hub.

“No direct communication between components — everything goes through API Server”

🔑 Key Concept

- 👉 API Server = Central Hub
 - All components talk to the API Server
 - Communication is secured using TLS & client certificates 🔒

🔑 Communication Flow

🧠 API Server ↔ etcd

- API Server communicates directly with etcd
 - Uses: TLS encryption 🔒 & Client certificates 📄
 - Purpose: Store cluster state & Retrieve data
 - 👉 Only API Server talks to etcd directly

⚙️ Controller Manager & Scheduler

generated by "Markdown to PDF Fast Converter" 🌟 <https://dub.sh/79wTuSy>

- Communicate with API Server (NOT etcd)
- Use:
 - Client certificates & HTTPS (TLS)
 - Watches cluster state & Makes changes via API Server
 - ✓ Example: Pod crashes → Controller recreates it

Kubelet ↔ API Server (Worker Nodes)

- Talks to API Server using: Mutual TLS (mTLS) 
- Ensures:
 - Secure node communication
 - Identity verification
 -  Why mTLS?
 - Both sides verify identity
 - Prevents unauthorized nodes

Security Architecture

TLS Security

- All communication is:
 - Encrypted 
 - Authenticated 
 - Secure (Zero Trust Model)

Certificate Authority (CA)

- Kubernetes has an internal CA
- Used to:
 - Issue certificates
 - Verify identities

Kubernetes Control Plane Security

Control Plane Communication :

-  API Server is central hub

- All components communicate via API Server securely
- API Server ↔ etcd → TLS encryption (etcd only talks to API Server)
- Scheduler & Controller Manager → API Server
- kubelet → API Server (mTLS authentication)

Kubernetes PKI (Certificates System)

Certificate Location

Certificates are stored in:

`/etc/kubernetes/pki/`

Kubernetes TLS Files (Core)

 File	 Purpose	 Explanation
 ca.crt	Root Certificate Authority	 Trust anchor used to verify all certificates in cluster
 apiserver.crt	API Server Certificate	 Identity of API Server (shared with clients)
 apiserver.key	API Server Private Key	 Secret key used by API Server for encryption

Simple Understanding

-  ca.crt → Trusted authority (who signs)
-  apiserver.crt → API Server ID card 
-  apiserver.key → Secret password 

Zero Trust Model

- Every component must authenticate
- No trust
- All communication is encrypted

Summary

-  API Server = central communication hub
 -  etcd = stores cluster state (Only API Server talks to etcd directly)
 -  TLS secures all communication
 -  mTLS used for node communication
 -  Zero Trust security model
 -  kubeadm auto-generates certs
-  Secure cluster = Reliable cluster  & Secure communication = Stable Kubernetes cluster 